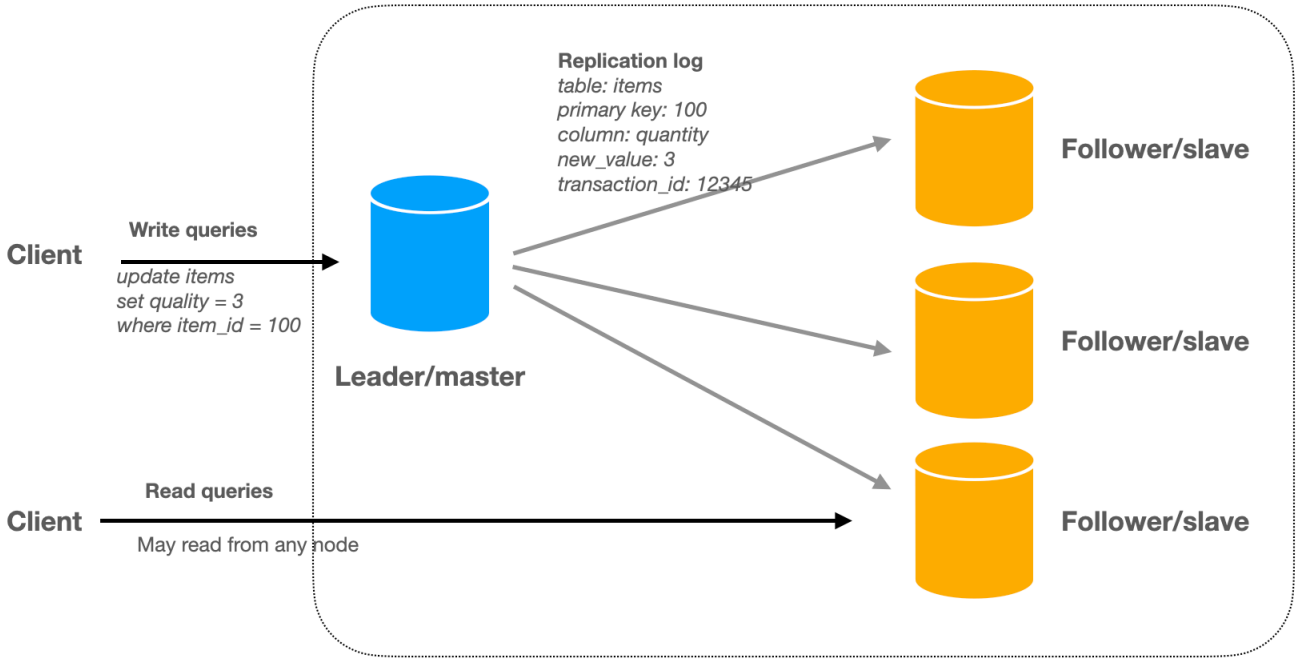
Database Replication in System Design

systemdesignschool.io/fundamentals/database-replication



Database Replication: Fundamentals and Algorithms

What is Replication

Make multiple copies of the data and put them on different machines (often called nodes like a node in a graph). Each node that stores a copy of the data is a **replica**.

Why Replication

The primary benefit is to **scale the system to handle more read requests**. Throughput: With multiple replicas, the system can handle more **read requests** simultaneously. Additionally, it also improves:

- Availability: The system continues to operate even when some replicas are down.
- Latency: By placing replicas close to the user's geographical location, the response time can be reduced.

Why Replication is Tricky

If the data is immutable, replication is straightforward: simply copy the data and it's done.

The challenge arises when dealing with mutable data, i.e., ensuring that all replicas consistently reflect the same data changes. This requires implementing strategies to synchronize and manage updates across replicas, while maintaining the desired level of consistency, availability, and performance. This may sound abstract, but it'll be very clear once we take a look at how replication algorithms work.

Replication Algorithms

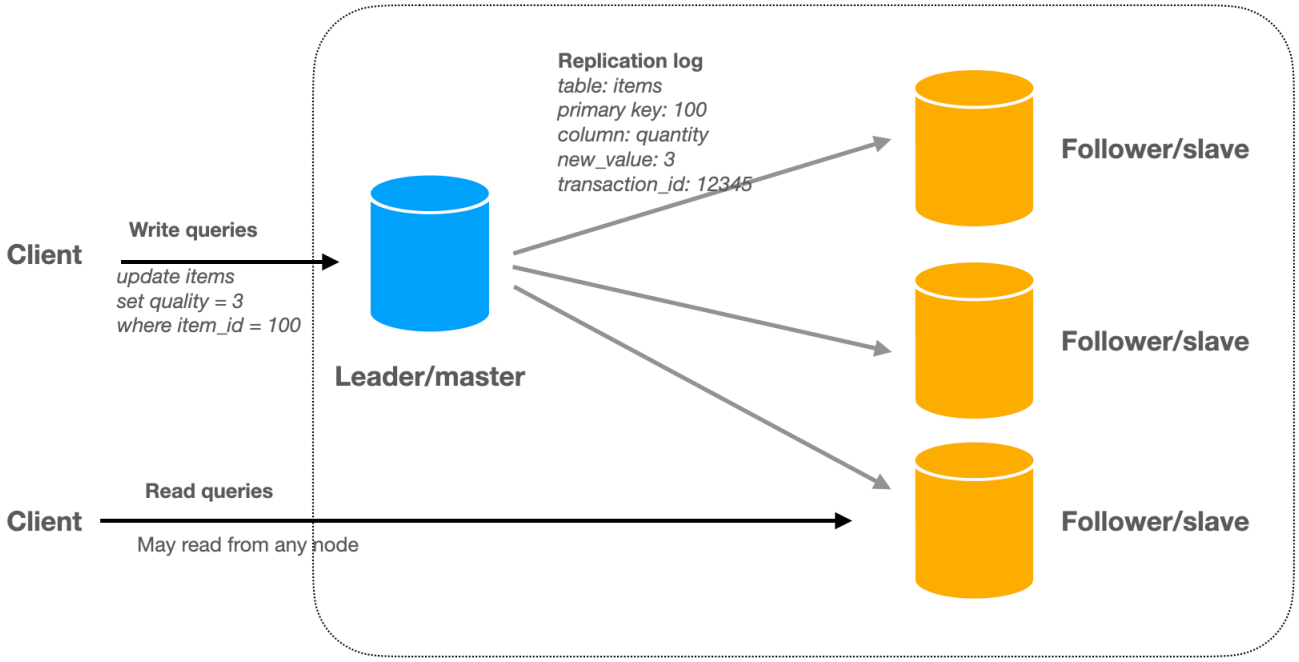
First let's clarify the terminology:

Leader = primary

Follower = secondary = replica

Single-Leader Replication

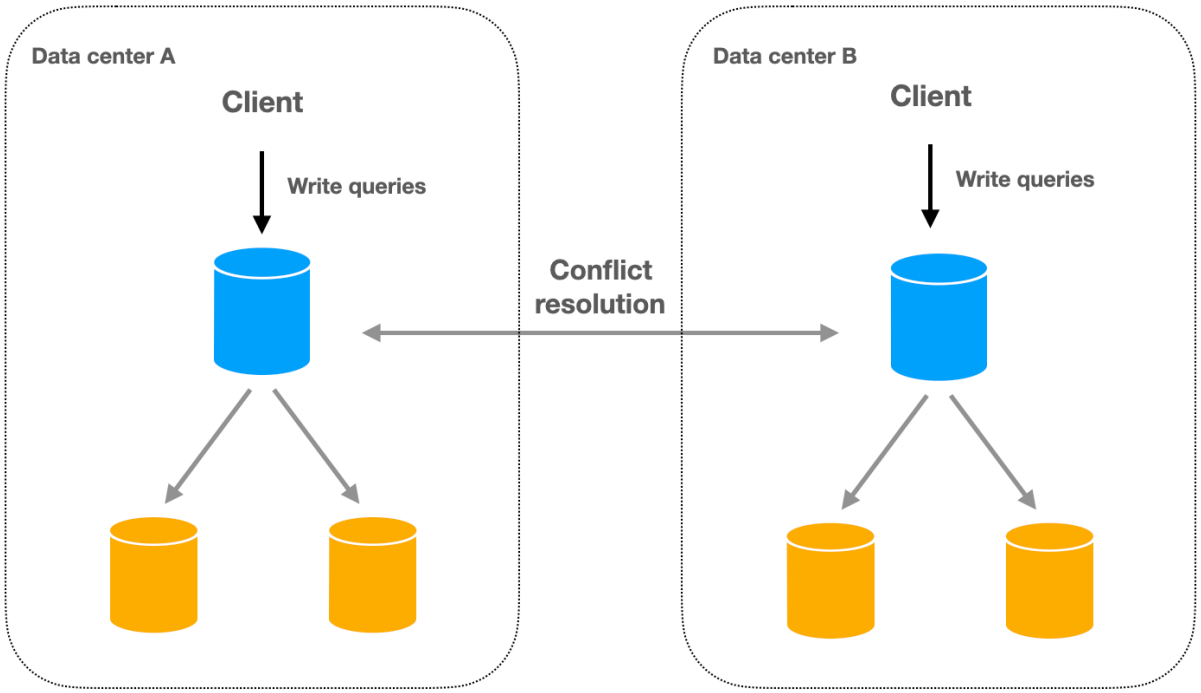
In single-leader replication, write operations are directed to a single leader (also known as the primary). The leader processes the write requests and then replicates the changes to its followers (also known as the replicas or secondaries). Read operations can be served by the leader or any of the followers. This approach is the most common replication method used by databases. We will cover the nitty-gritty of single-leader replication later in this article.



Multi-Leader Replication

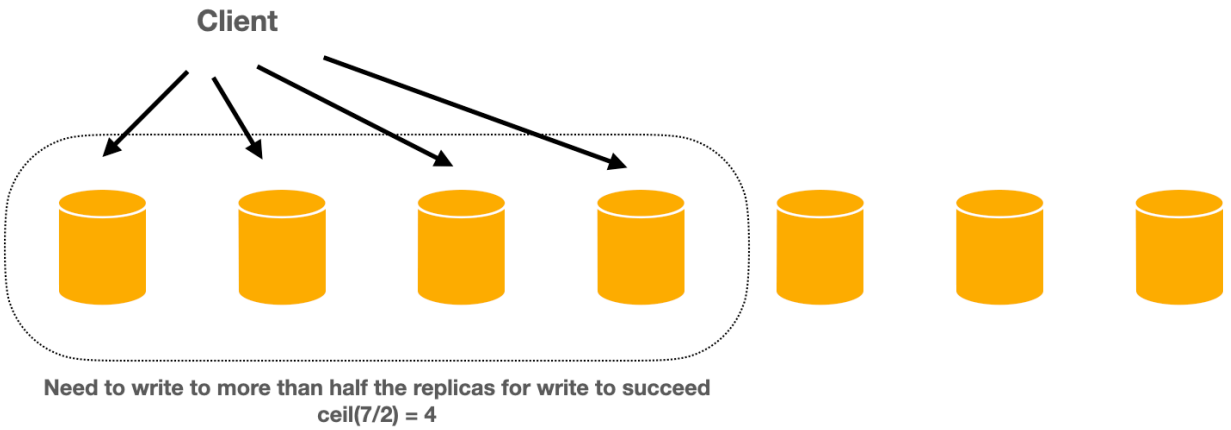
There are multiple leaders, and clients can send requests to any of them. The leaders send data changes to each other and followers.

Multi-leader replication is mainly used in setups with multiple data centers, where each data center has its own leader. Write operations can be performed on any leader, and changes are then propagated to other leaders and their followers. This approach introduces the challenge of handling conflicts that may arise when different leaders accept conflicting updates for the same data. Conflict resolution mechanisms must be put in place to ensure data consistency across all replicas.



Leaderless Replication

In leaderless replication, write operations can be directed to any replica, and a quorum is used to decide whether read and write operations succeed. This means that a certain number of replicas must agree on the state of the data before an operation is considered successful. Leaderless replication is a unique approach that is mostly used in Dynamo-style databases like the original Amazon Dynamo from 2007, Apache Cassandra, and Riak.


Client
Need to write to more than half the replicas for write to succeed
ceil(7/2) = 4